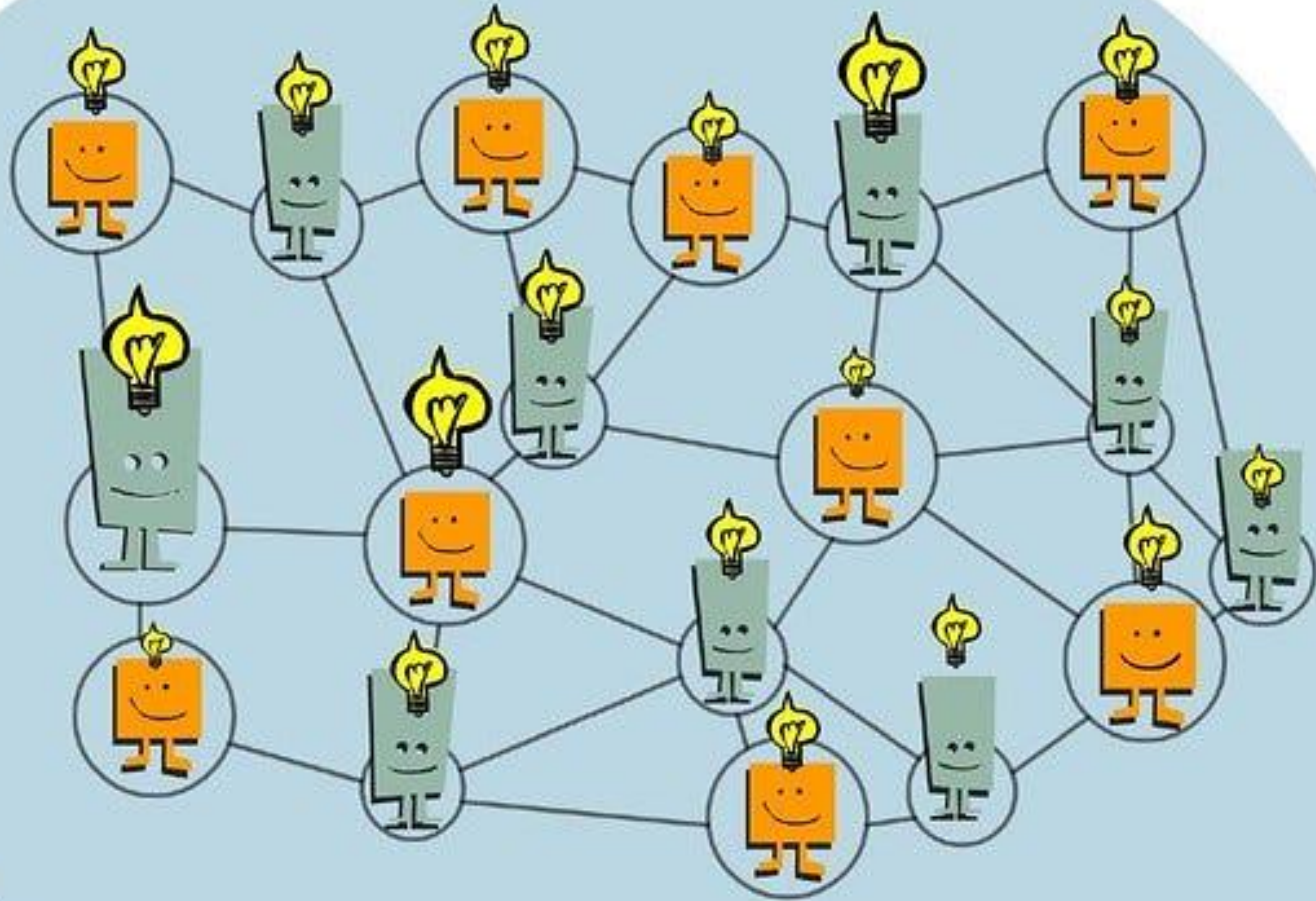


# Desempeño y Optimización para la Latencia



**CADA UNO SOSTIENE  
UNA PARTE**

# Desempeño y Optimización para la Latencia

El desempeño y la optimización de la latencia en un sistema distribuido se refieren a cómo se mide y mejora la rapidez con la que los nodos del sistema responden a solicitudes, reduciendo retrasos en la comunicación y garantizando eficiencia en el procesamiento. En otras palabras, se busca que el sistema distribuido sea rápido, escalable y confiable, minimizando el tiempo de espera de los usuarios

# Desempeño o Performace.

El desempeño en un sistema distribuido se refiere a la capacidad del sistema para ejecutar tareas de manera eficiente y rápida, aprovechando múltiples nodos conectados en red para mejorar la velocidad, la escalabilidad y la tolerancia a fallos. En otras palabras, mide qué tan bien funciona el sistema al repartir la carga de trabajo entre varias computadoras

# Desempeño o Performace.

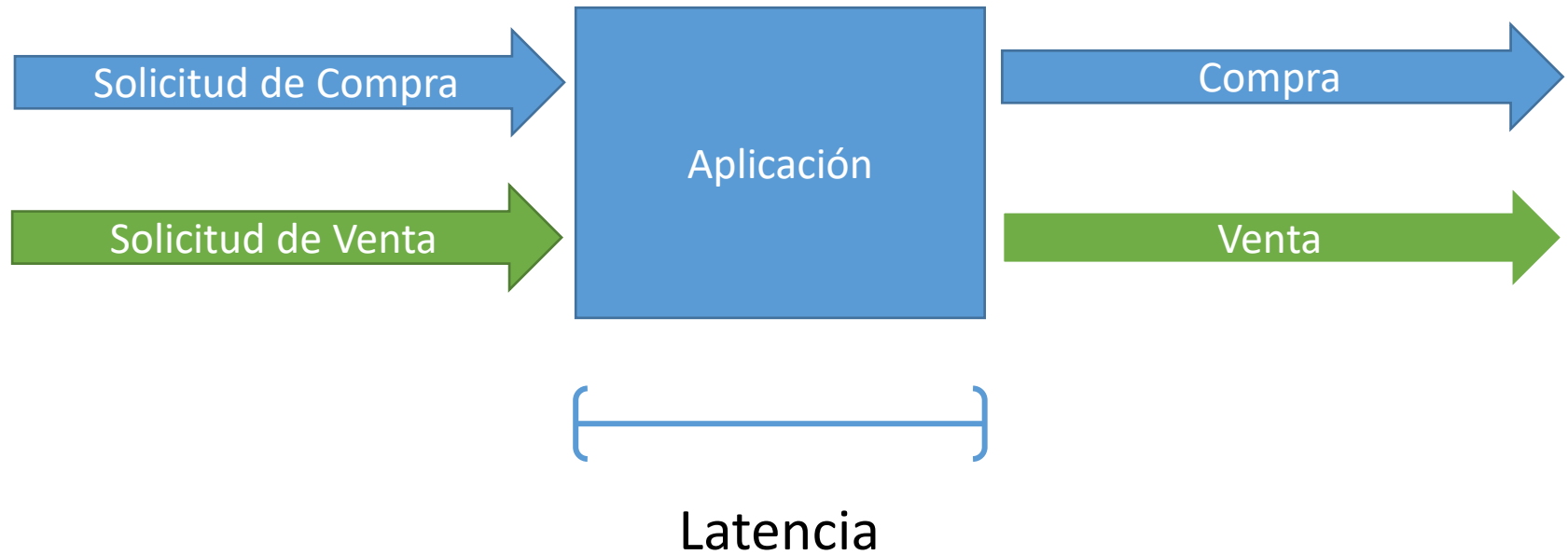
Desempeño se refiere a qué tan eficiente o efectiva es una operación, sistema o tecnología al realizar sus funciones. Este término generalmente abarca aspectos como:

- **Velocidad de procesamiento:** Qué tan rápido puede completarse una tarea o proceso.
- **Uso de recursos:** La cantidad de recursos utilizados (como CPU, memoria, energía) para realizar una operación.
- **Confiabilidad:** Consistencia en la entrega de resultados sin fallos.
- **Escalabilidad:** La capacidad del sistema de manejar un aumento en la carga o demanda sin degradar su desempeño.

# Latencia

- La latencia en un sistema distribuido es el tiempo que tarda una solicitud en viajar desde un nodo de origen hasta un nodo de destino y regresar con la respuesta. Es una medida crítica porque determina la rapidez con la que los usuarios perciben que el sistema responde. Una latencia baja significa respuestas rápidas; una latencia alta implica demoras y peor experiencia de usuario.
- Es el tiempo que tarda una acción en generar una respuesta, y puede ser afectada por factores como la distancia física, la congestión de la red, las limitaciones de hardware y las ineficiencias de software

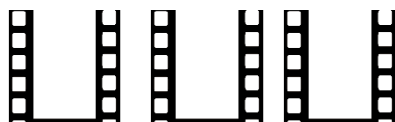
# Sistema de Comercio de Alta Velocidad



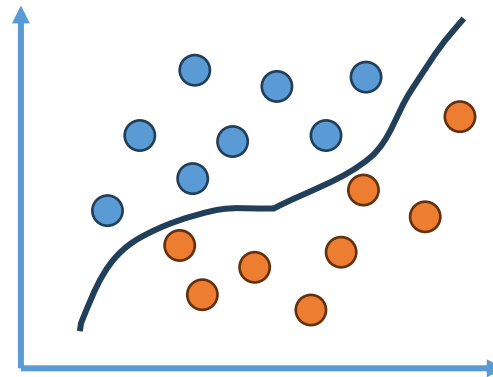
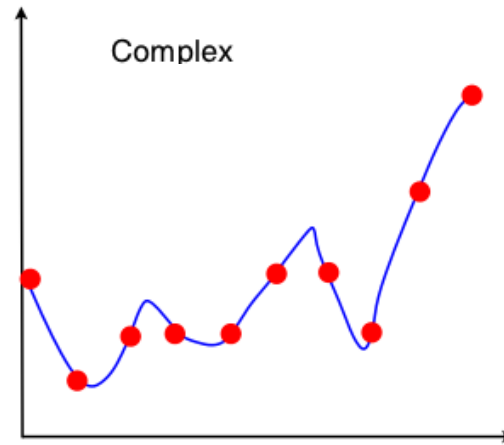
# Optimizar la latencia

- Implica identificar y abordar cuellos de botella en componentes como el retraso de propagación, el retraso de transmisión, el retraso de procesamiento y el retraso en las colas de espera.
- Esto puede lograrse mediante el uso de algoritmos eficientes, hardware actualizado y estrategias de diseño que minimicen la distancia y la congestión en la red.

# Reproductor de Video



# Machine Learning



# Rendimiento en subprocesos múltiples

- Throughput (Rendimiento): la cantidad de operaciones o tareas completadas en un período determinado. Medido en **unidades de tareas/tiempo**, por ejemplos:
  - Operaciones por segundo (ops/s), por ejemplo: una base de datos distribuida que procesa 10,000 consultas por segundo.
  - Transacciones por segundo (TPS), por ejemplo, usado en sistemas financieros o de comercio electrónico.
  - Solicitudes por segundo (req/s), común en servidores web y APIs distribuidas.
  - Bytes por segundo (B/s), Kilobytes por segundo (KB/s), Megabytes por segundo (MB/s), por ejemplo, usado en sistemas de transmisión de datos o almacenamiento distribuido.
  - Mensajes por segundo, por ejemplo, usado en sistemas de mensajería distribuida (como Kafka o RabbitMQ).

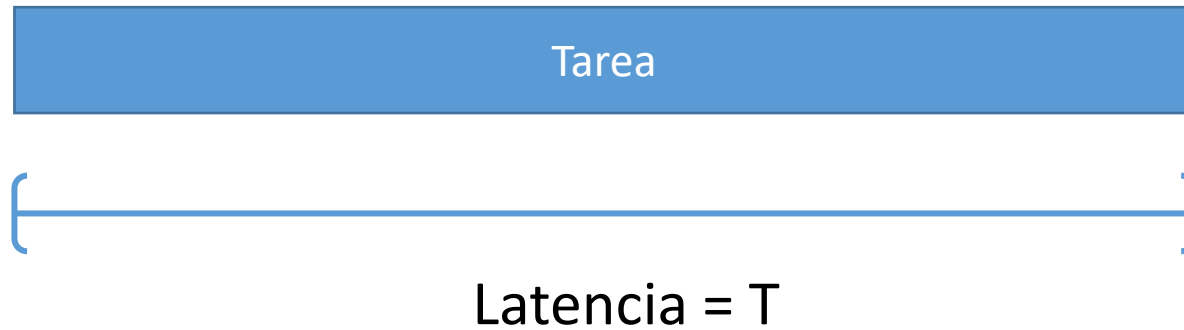
# Rendimiento en subprocesos múltiples

El rendimiento en subprocesos múltiples se refiere a la eficiencia con la que un programa aprovecha varios hilos de ejecución simultáneamente para mejorar la velocidad, la capacidad de respuesta y la escalabilidad.

En esencia, mide cuánto más rápido y fluido puede funcionar una aplicación cuando divide su trabajo en múltiples hilos que corren en paralelo dentro de uno o varios núcleos de CPU.

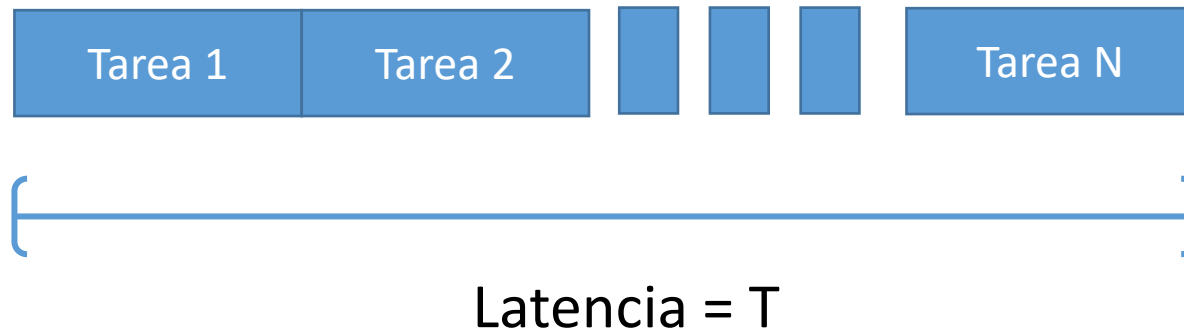
- Latencia: El tiempo para completar una tarea. Medido en unidades de tiempo, por ejemplo, milisegundos (ms) en algunos casos microsegundos ( $\mu$ s) o incluso nanosegundos (ns)
- Throughput (Rendimiento): la cantidad de operaciones o tareas completadas en un período determinado. Medido en unidades de tareas/tiempo, por ejemplos

# Latency (latencia)



T: una tarea

# Latency (latencia)



T: una tarea

Ti: múltiples tareas independientes

# Latency (latencia)



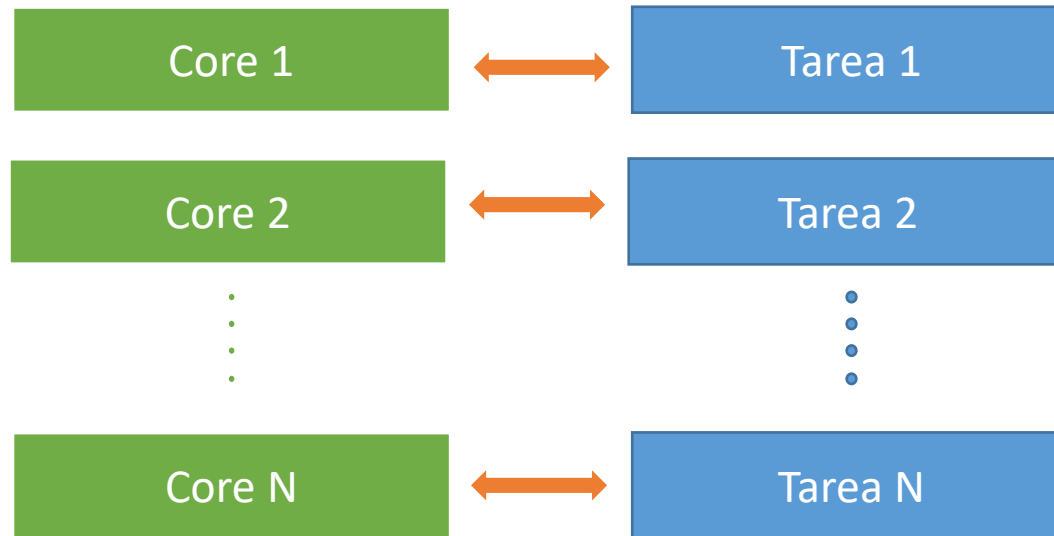
N: numero de sub tareas

# Latency (latencia)

- Es un enfoque teórico para reducir la latencia en un factor  $N$ , y mejorar el rendimiento en un factor de  $N$ .
- $N=?$ 
  - ¿Cuántas subtareas/hilos podemos tener, para poder dividir la tarea original?
- ¿Podemos dividir cualquier tarea en subtareas?

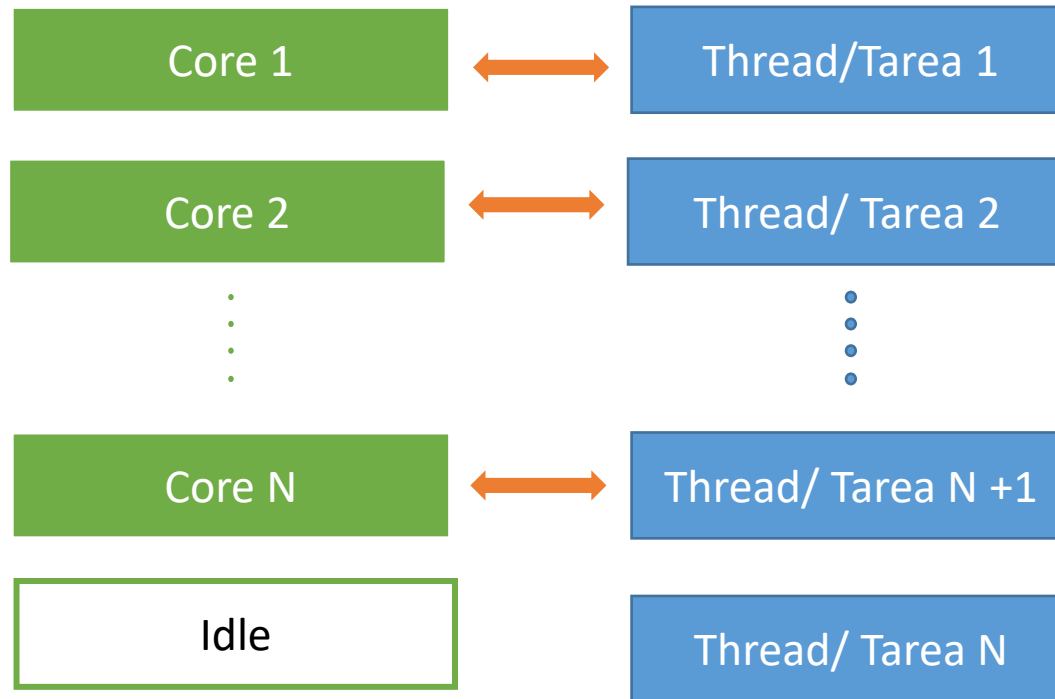
$N=?$

- En una computadora de propósito general:  $N$  = numero de núcleos



N=?

- Con un hilo extra y tareas =  $N + 1$



N=?

- #threads = # cores es óptimo solo si todos los subprocesos son ejecutables y pueden ejecutarse sin interrupción (sin I/O/bloqueo de llamadas/sleep, etc.)
- La suposición es que no se está ejecutando nada más que consuma una gran cantidad de CPU

# ¿Cuál es la diferencia entre Hyper Threading y la tecnología Multi-Core?

## Hyper-Threading Technology

- La tecnología Hyper-Threading crea dos núcleos de procesamiento virtual para cada núcleo físico presente en una CPU. El **physical core (núcleo físico)** alimenta los **virtual cores (núcleos virtuales)**, que luego comparten la responsabilidad del procesamiento de tareas. Cada núcleo virtual es idéntico al otro y, aunque ninguno es tan potente como el núcleo físico, juntos superan con creces la potencia del núcleo físico

# ¿Cuál es la diferencia entre Hyper Threading y la tecnología Multi-Core?

## Multi-Core Technology

- La tecnología multinúcleo, que suele estar disponible en CPU de dos núcleos, cuatro núcleos y hexa-núcleos, es una tecnología que agrega núcleos de procesamiento físico adicionales. En una CPU de un solo núcleo, las tareas se procesan una a la vez por orden de llegada. Esto puede ser problemático para la multitarea, ya que las tareas comenzarán a respaldarse. En un procesador con dos o más núcleos, la multitarea es mucho más eficiente ya que hay múltiples núcleos disponibles para procesar tareas. Cuantos más núcleos tenga, más datos podrá procesar sin que disminuya el rendimiento.

# Simultaneous Multithreading (SMT),

Simultaneous Multithreading (SMT), conocido comúnmente como Hyper-Threading en Intel o SMT en AMD, es una tecnología de procesador que permite a un solo núcleo físico gestionar dos hilos (o tareas) simultáneamente. Esto mejora la eficiencia al aprovechar recursos de la CPU que, de otro modo, estarían inactivos, aumentando el rendimiento en multitarea, características sobre SMT:

- **Funcionamiento:** En lugar de alternar entre hilos, el procesador emite instrucciones de múltiples hilos en el mismo ciclo de reloj.
- **Capacidad Lógica:** Un procesador de 4 núcleos y 8 hilos significa que cada núcleo físico ejecuta dos tareas a la vez.
- **Uso ideal** para renderizado 3D, edición de video y multitarea pesada.
- **Rendimiento:** Aunque no duplica la velocidad, puede ofrecer mejoras significativas (hasta 70-80% en algunos casos) en software especializado.

# Diferencia entre Simultaneous Threading vs Multithreading

La principal diferencia es que el Multithreading (Multihilo) es un concepto general de software que gestiona múltiples tareas (hilos) compartiendo recursos para mejorar la eficiencia, mientras que el Simultaneous Multithreading (SMT) es una técnica de hardware específica (como Hyper-Threading de Intel) que permite a un solo núcleo físico ejecutar dos hilos simultáneamente para maximizar el uso de la CPU.

## Impacto y Propiedad:

- Intel llama a su implementación SMT "Hyper-Threading".
- AMD utiliza SMT en sus procesadores Ryzen/EPYC.
- Ambos convierten núcleos físicos en dos núcleos lógicos.

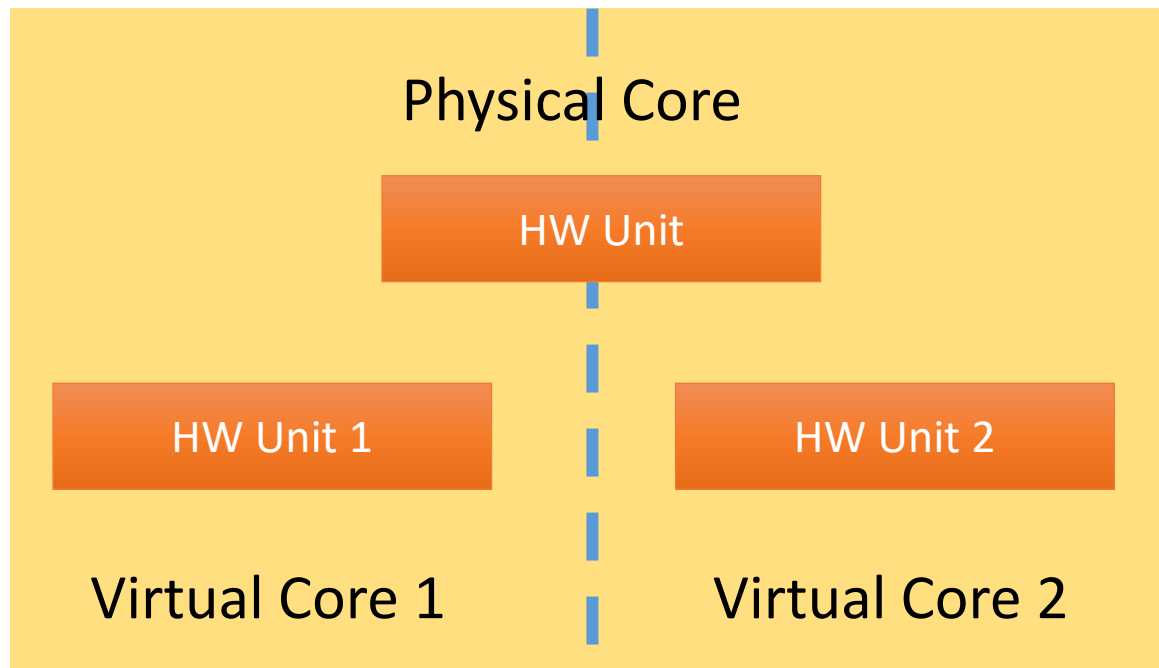
# Diferencia entre Simultaneous Threading vs Multithreading

## Diferencias Clave:

- Multithreading (General/Software): Un proceso se divide en hilos que comparten la misma memoria. El sistema operativo alterna entre ellos (a veces simulando simultaneidad) para mejorar la capacidad de respuesta, ideal para tareas I/O-bound (esperas de red/disco).
- Simultaneous Multithreading (Hardware - SMT): El procesador mantiene el contexto de hardware de dos hilos. SMT aprovecha los ciclos inactivos de un núcleo para ejecutar instrucciones de diferentes hilos en el mismo ciclo, logrando un verdadero paralelismo a nivel de ejecución.

N=?

- Hyper Threading – Virtual Cores vs Physical Cores

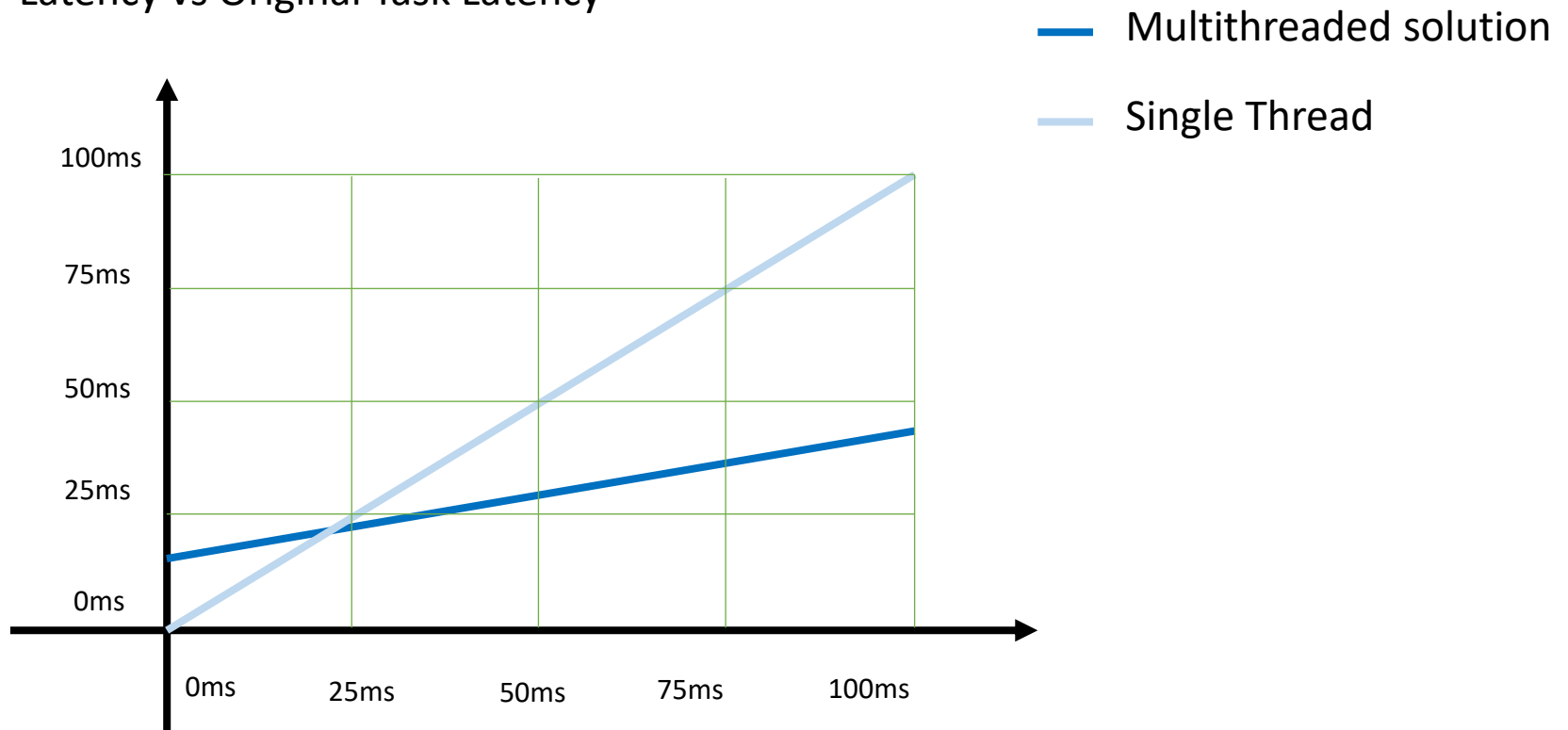


# Costo inherente de paralelización y agregación

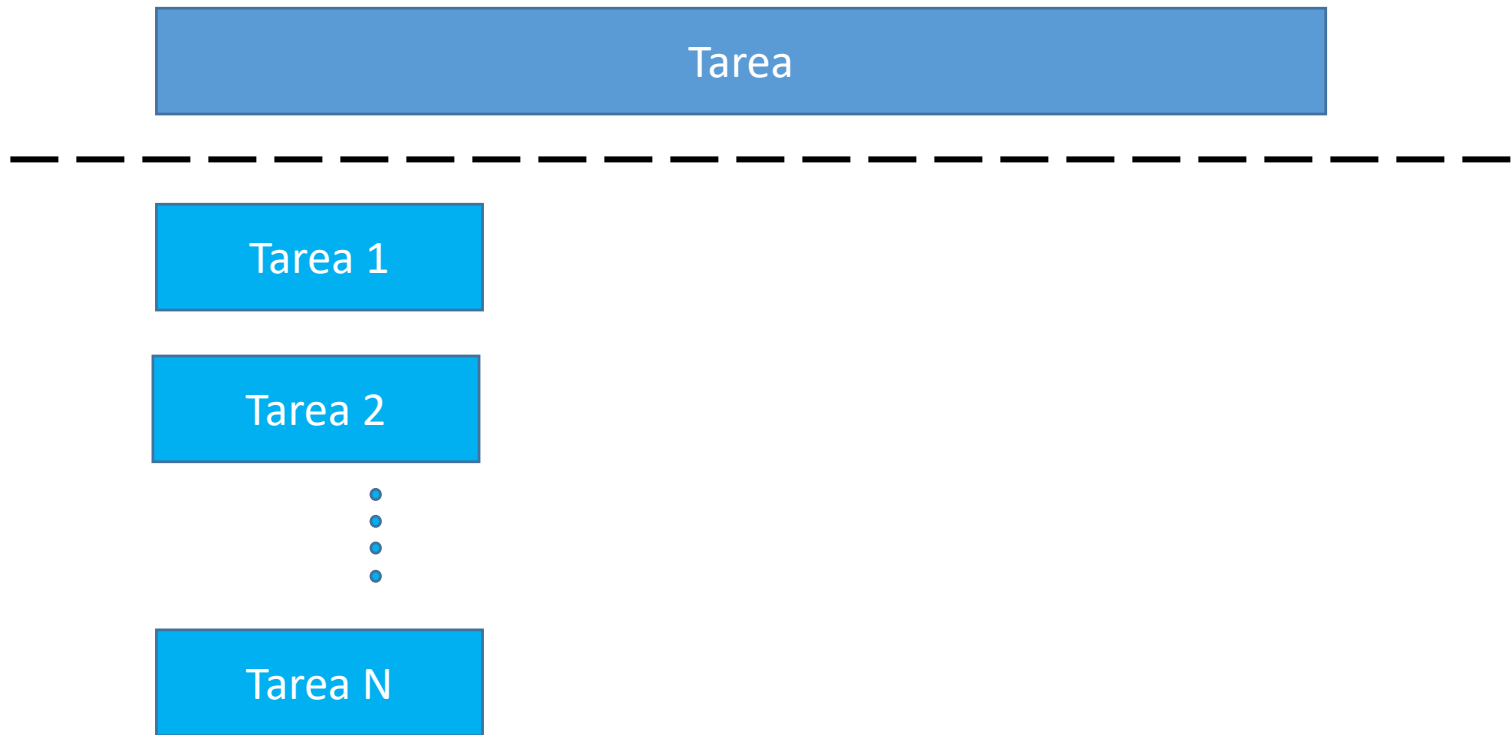
- Dividir la tarea en múltiples tareas.
- Creación de hilos, pasando tareas a hilos.
- Tiempo entre `thread.start()` para programar el hilo.
- Tiempo hasta que finaliza el último hilo y señala.
- Tiempo hasta que se ejecuta el subproceso de agregación.
- Agregación de los subresultados en un solo artefacto.

# Costo inherente de paralelización y agregación

Latency vs Original Task Latency

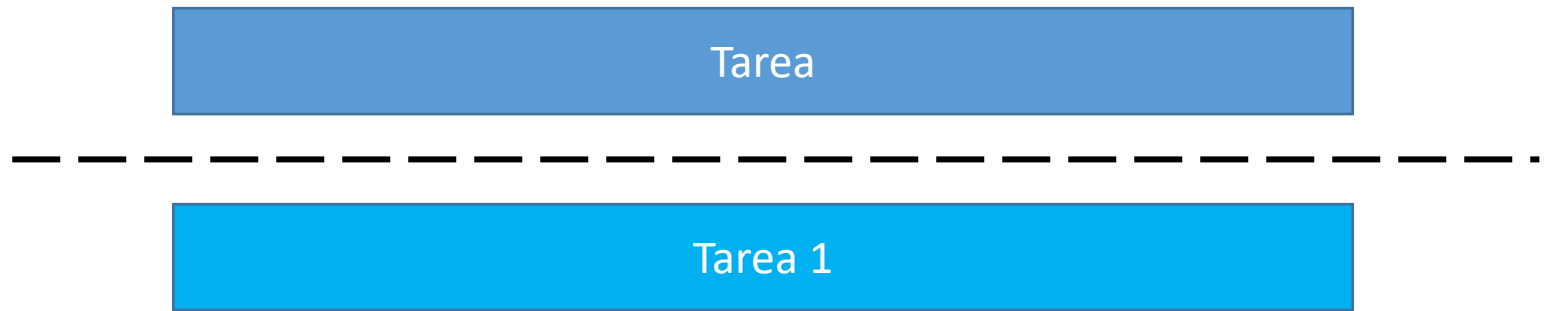


# Tipo 1 Paralelización de Tareas



Varios hilos, simultaneos

## Tipo 2: tarea secuencial irrompible



Un Hilo, secuencial

## Tipo 3 - Parcialmente Paralelizable, Parcialmente Secuencial

